

Practical 10

Task 1: Investigate inode structures using commands such as `ls -li`, `stat`, and `find`. Create hard links and symbolic links and explain their impact on inode allocation.

Code:

```
rohita@DESKTOP-1UHU58C: ~
GNU nano 8.7.1 p10.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/stat.h>
#include <sys/types.h>

void print_inode_info(const char *filename) {
    struct stat file_stat;

    if (lstat(filename, &file_stat) == -1) {
        perror("lstat failed");
        return;
    }

    printf("File: %-20s | Inode: %-10lu | Hard Links: %-2ld | Size: %-5ld bytes\n",
        filename, file_stat.st_ino, file_stat.st_nlink, file_stat.st_size);
}

int main() {
    const char *original = "original.txt";
    const char *hardlink = "hardlink.txt";
    const char *symlink_file = "symlink.txt";

    FILE *fp = fopen(original, "w");
    if (!fp) {
        perror("Failed to create base file");
        return 1;
    }
    fputs("Hello, inode structure!", fp);
    fclose(fp);

    printf("--- Initial File Creation ---\n");
    print_inode_info(original);

    if (link(original, hardlink) != 0) {
        perror("Failed to create hard link");
        return 1;
    }

    printf("\n--- After Creating Hard Link ---\n");
    print_inode_info(original);

    print_inode_info(hardlink);

    if (symlink(original, symlink_file) != 0) {
        perror("Failed to create symbolic link");
        return 1;
    }

    printf("\n--- After Creating Symbolic Link ---\n");
    print_inode_info(original);
    print_inode_info(hardlink);
    print_inode_info(symlink_file);

    unlink(original);
    unlink(hardlink);
    unlink(symlink_file);
}
```

Output:

```
rohita@DESKTOP-1UHU58C:~$ nano p10.c
rohita@DESKTOP-1UHU58C:~$ gcc p10.c
rohita@DESKTOP-1UHU58C:~$ ./a.out
--- Initial File Creation ---
File: original.txt      | Inode: 46775      | Hard Links: 1 | Size: 23  bytes

--- After Creating Hard Link ---
File: original.txt      | Inode: 46775      | Hard Links: 2 | Size: 23  bytes
File: hardlink.txt      | Inode: 46775      | Hard Links: 2 | Size: 23  bytes

--- After Creating Symbolic Link ---
File: original.txt      | Inode: 46775      | Hard Links: 2 | Size: 23  bytes
File: hardlink.txt      | Inode: 46775      | Hard Links: 2 | Size: 23  bytes
File: symlink.txt       | Inode: 46777      | Hard Links: 1 | Size: 12  bytes
```

Observation The inode information of files was examined using `ls -li`, `stat`, and `find`. Hard links shared the same inode, while symbolic links had a different inode and pointed to the original file.

Task 2: Design a program using `mmap()` to perform file reading and writing. Compare memory-mapped I/O with traditional read/write operations in terms of efficiency and implementation complexity.

Code:

```

rohita@DESKTOP-1UHU58C: ~
GNU nano 8.7.1
#include <stdio.h>
#include <fcntl.h>
#include <sys/mman.h>
#include <unistd.h>
#include <string.h>

int main() {
    int fd;
    char *data;
    const char *text = "Hello, mmap!";

    fd = open("file.txt", O_RDWR | O_CREAT, 0666);

    if (fd == -1) {
        perror("open");
        return 1;
    }

    ftruncate(fd, strlen(text) + 1);

    data = mmap(NULL, strlen(text) + 1,
                PROT_READ | PROT_WRITE,
                MAP_SHARED, fd, 0);

    if (data == MAP_FAILED) {
        perror("mmap");
        close(fd);
        return 1;
    }

    strcpy(data, text);

    printf("Data written: %s\n", data);

    munmap(data, strlen(text) + 1);
    close(fd);

    return 0;
}

```

Output:

```

rohita@DESKTOP-1UHU58C:~$ ./a.out
Data written: Hello, mmap!
rohita@DESKTOP-1UHU58C:~$ _

```

Observation: The `mmap()` system call was used to read and write a file through memory mapping. Memory-mapped I/O provides faster and simpler access to file data compared to traditional `read()` and `write()` operations.